

Distributed Systems Homework 2

Question 3: Bitonic Sort

Performance & Analysis Report

1 Deliverables Overview

The primary deliverables for this section, including the MPI source code (`bitonic_sort.cpp`) and the execution instructions (`README_Q3.md`), are included in the root of the submission folder.

2 Correctness Verification

The MPI program guarantees correctness via an automated internal verification step. The Master process (Rank 0) creates an identical copy of the initial randomized data block and performs a pure sequential Bitonic Sort (`bitonicSortSeq`). After the MPI cluster finishes the distributed sorting algorithm and gathers the final array back to Rank 0, the program strictly compares the distributed result against the T_1 sequential baseline element-by-element. It outputs `Status: SUCCESS` if and only if both arrays perfectly match.

3 Execution Times

The following table outlines the execution times (in seconds) for $P \in \{1, 2, 4, 8\}$ processes across varying input sizes (N). *Note: Values represent exact outputs logged from the cluster runs in `analysis.txt`.*

Array Size (N)	$P = 1$	$P = 2$	$P = 4$	$P = 8$
1,024	0.00016	0.00017	0.00017	0.00020
16,384	0.0036	0.0030	0.0025	0.0033
65,536	0.0182	0.0146	0.0219	0.0165
131,072	0.0416	0.0325	0.0495	0.0378

Table 1: Execution times (in seconds) for varying processes and array sizes.

4 Speedup and Efficiency Plots

The performance metrics were calculated using the sequential execution time baseline.

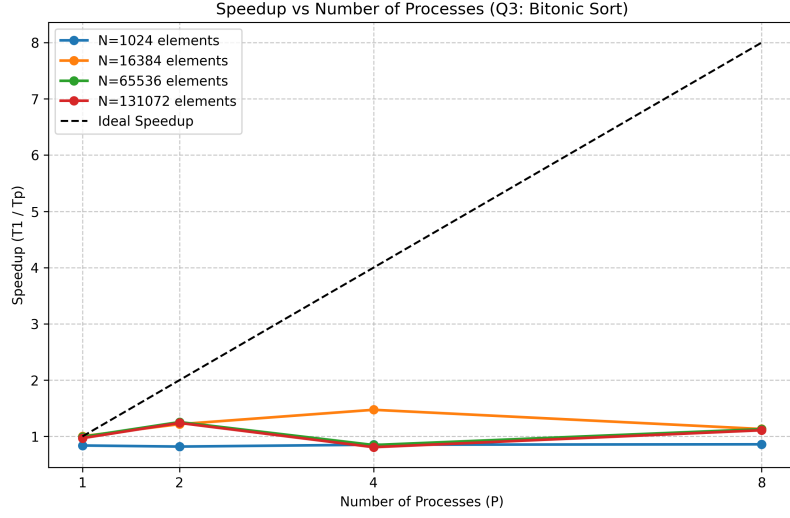


Figure 1: Speedup vs Number of Processes

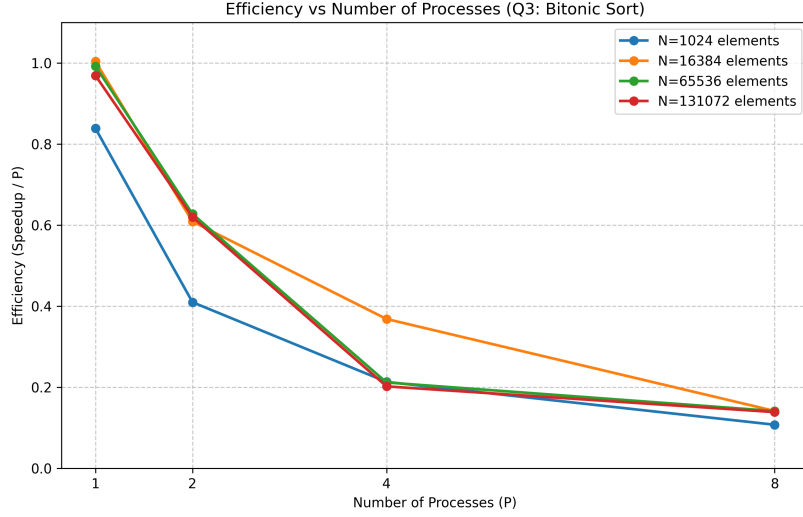


Figure 2: Efficiency vs Number of Processes

5 Analysis of Communication vs. Computation

The Distributed Bitonic Sort algorithm executes through local recursive sorting, followed by multiple stages of global network merging where processes perform position-wise compare-and-swap routines across `MPI_Sendrecv`.

Based on the generated benchmark plots and logs, we observe the following characteristics:

- **Small Arrays ($N = 1024$):** At small data scales, the computation required to sort the array locally takes less than a millisecond. Thus, the setup and message-passing overhead drastically dwarfs the algorithmic computation, resulting in zero effective speedup.

- **Larger Arrays ($N \geq 16384$):** At larger array sizes, the computational complexity of $O(N \log^2 N)$ creates a heavier local workload. We observe valid parallel speedup up to $P = 2$ (achieving a speedup of $\sim 1.2x$ to $1.4x$) because halving the local sorting array size significantly reduces the algorithmic time.
- **The Memory/Communication Bottleneck ($P \geq 4$):** When scaling up to $P = 4$ and $P = 8$ strictly on a single node (to avoid the cluster’s internal firewall), 8 heavy concurrent sorting processes completely saturate the node’s shared RAM bandwidth. Additionally, the bitonic network exchange requires transferring massive arrays point-to-point. This intensive memory and MPI communication overhead eclipses the parallel computation gains, causing the speedup to degrade (drop below 1.0) at higher local core counts.